Introduction aux Bases de Données Relationnelles et Requêtes SQL (Norme ISO/ANSI)

Thème principal : Modèle Relationnel, Intégrité Référentielle et Interrogation de Données via SQL ANSI

Synthèse exécutive : Ce cours fondamental de la ressource R1.05 pose l'intégralité des bases théoriques et pratiques de la gestion de données en informatique. Il justifie pourquoi l'industrie abandonne les fichiers plats et les tableurs (type Excel) au profit des Systèmes de Gestion de Bases de Données Relationnelles (SGBD) afin d'éliminer la redondance d'information et les anomalies de mise à jour. Le cours détaille la modélisation sous forme de tables reliées par des clés primaires (garantissant l'unicité et l'existence) et des clés étrangères (matérialisant les liens), maintenues par la contrainte d'intégrité référentielle. Enfin, il enseigne la rédaction de requêtes SQL d'extraction fiables et optimisées selon la norme ISO/ANSI, en imposant l'usage strict des jointures explicites (INNER JOIN ... ON) et la proscription du produit cartésien implicite.

Du Tableur Plat au Modèle Relationnel : Nécessité et Enjeux

Dans la conception d'applications informatiques, le stockage des informations sous forme de tableurs plats (fichiers CSV ou feuilles Excel) se heurte rapidement à des limites structurelles rédhibitoires. Dans un tableur plat, si un client passe 50 commandes et que son adresse postale est répétée sur chaque ligne de commande, toute modification d'adresse exige 50 opérations de mise à jour distinctes. Si une seule ligne est manquée, le système entre dans un état incohérent : c'est l'anomalie de mise à jour. Le modèle relationnel résout ce problème à la racine en appliquant le principe de séparation des responsabilités : on crée une table dédiée aux 'Clients' et une table dédiée aux 'Commandes'. L'adresse du client n'est présente qu'une seule fois dans la table 'Clients'. La table 'Commandes' se contente d'établir un pointeur logique vers le client concerné. Ainsi, un déménagement ne nécessite qu'une unique modification dans la table client, garantissant l'intégrité absolue du système d'information.

- Fichier Plat / Tableur : Stockage bidimensionnel unique où toutes les informations sont alignées sans séparation logique des entités.
- Redondance de Données : Duplication inutile d'une même information à plusieurs endroits de la base, entraînant un gaspillage de stockage et un risque majeur d'incohérence.
- Anomalie de Mise à Jour : Incohérence survenant lorsqu'une donnée dupliquée est modifiée à un endroit mais oubliée à un autre.
- SGBD (Système de Gestion de Bases de Données) : Logiciel d'infrastructure garantissant l'intégrité, la sécurité, l'accès concurrentiel et la persistance des données.
- Modèle Relationnel : Organisation de l'information en entités distinctes (tables ou relations) interconnectées par des identifiants logiques.

Cas concret enseignant : *L'enseignant a illustré cette problématique avec l'analogie du système d'information bancaire : si une banque enregistrait le nom, le prénom et l'adresse du titulaire sur chaque ligne de transaction bancaire (virement, retrait, paiement par carte), un simple déménagement exigerait de repasser sur des milliers de lignes d'historique passées. Cela saturerait les serveurs et créerait un risque critique d'erreur de saisie.*

Clés Primaires, Clés Étrangères et Intégrité Référentielle

Pour qu'une base de données relationnelle fonctionne de façon déterministe, chaque table doit posséder une Clé Primaire. Celle-ci constitue le pivot d'identification absolue d'un n-uplet (une ligne). Une clé primaire ne peut jamais contenir la valeur NULL et ne peut comporter aucun doublon. Pour lier deux tables (par exemple rattacher un Étudiant à son Groupe de TD), on n'insère pas les attributs du groupe (nom du groupe, promotion, responsable) dans la table Étudiant. On insère uniquement la clé primaire du groupe (ex: id_groupe) au sein de la table Étudiant. Dans la table Étudiant, cet attribut devient une Clé Étrangère. Le SGBD surveille en permanence la contrainte d'intégrité référentielle : il est strictement impossible d'insérer un étudiant avec un id_groupe qui n'existe pas encore

dans la table Groupe, ou de supprimer un groupe si des étudiants y sont encore rattachés (sauf configuration explicite de suppressions en cascade).

• Clé Primaire (Primary Key - PK) : Attribut (ou ensemble d'attributs) identifiant chaque ligne (n-uplet) de façon unique. Elle doit obligatoirement respecter deux contraintes : Unicité stricte et Non-Nullité (NOT NULL).

• Clé Étrangère (Foreign Key - FK) : Attribut présent dans une table qui fait référence directe à la clé primaire d'une autre table pour matérialiser une association.

• Intégrité Référentielle : Règle métier automatisée par le SGBD qui interdit à une clé étrangère de pointer vers une valeur de clé primaire inexistante dans la table mère.

Cas concret enseignant : *L'enseignant a cité l'exemple du Numéro de Sécurité Sociale (NIR) ou de l'Identifiant Étudiant à l'IUT : même si deux étudiants portent exactement les mêmes nom et prénom au Pays Basque (ex: deux Jean Dupont à Bayonne), leur identifiant unique empêche toute collision ou confusion au sein de la base de données de l'université.*

Extraction de Données en SQL ANSI : Projection, Filtrage et Jointures Explicites

Le Structured Query Language (SQL) est le langage standard d'interrogation. Pour croiser des informations réparties dans plusieurs tables, la norme ISO/ANSI impose une syntaxe explicite via la clause `INNER JOIN table2 ON condition_de_jointure`. Historiquement, les requêtes multi-tables étaient écrites en listant les tables séparées par des virgules dans la clause `FROM` et en mettant la condition de jointure dans la clause `WHERE` (`FROM Etudiant, Groupe WHERE Etudiant.id_groupe = Groupe.id_groupe`). Cette approche par produit cartésien implicite est désormais proscrite. La syntaxe ISO/ANSI moderne sépare nettement la structure relationnelle (dans le `JOIN ... ON`) du filtrage des lignes métier (dans le `WHERE`), améliorant la lisibilité, la maintenabilité et l'optimisation des requêtes par le moteur SQL.

• Langage Déclaratif : Le SQL exprime CE QUE l'on souhaite obtenir et laisse le SGBD déterminer algorithmiquement COMMENT l'extraire.

• Projection (SELECT) : Spécification explicite des colonnes à afficher dans le jeu de résultats.

• Filtrage (WHERE) : Application de critères logiques pour sélectionner les n-uplets d'intérêt.

• Jointure Explicite (INNER JOIN ... ON) : Combinaison normalisée ISO/ANSI de deux tables basée sur la correspondance exacte entre une clé étrangère et une clé primaire.

Cas concret enseignant : *Avertissement strict de l'enseignant pour les contrôles et les projets : 'N'utilisez jamais la syntaxe obsolète du produit cartésien avec une virgule dans le FROM (ex: FROM Etudiant, Groupe WHERE Etudiant.id_g = Groupe.id). Utilisez TOUJOURS la syntaxe explicite INNER JOIN ... ON. Le produit cartésien implicite fait perdre des points en partiel et est formellement interdit en entreprise.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne doit-on jamais utiliser 'SELECT *' dans un code applicatif ou lors d'un examen ?
Réponse du professeur : L'utilisation de 'SELECT *' est une mauvaise pratique majeure pour trois raisons : 1) Performance : elle force le SGBD à transférer des colonnes inutilement via le réseau. 2) Fragilité applicative : si la structure de la table évolue (ajout, suppression ou réordonnancement de colonnes), le code applicatif qui s'attend à un ordre précis d'attributs plante. 3) Évaluation : en examen, le professeur évalue votre capacité à réaliser une projection précise des seuls attributs demandés par l'énoncé.

Question étudiant : Que se passe-t-il si l'on tente de supprimer un groupe qui contient encore des étudiants rattachés ?
Réponse du professeur : Le SGBD intercepte l'opération et lève une erreur de violation de contrainte d'intégrité référentielle. Par défaut, la suppression du groupe parent est bloquée pour éviter d'avoir des étudiants orphelins (pointant vers un groupe inexistant). Pour autoriser la suppression, il faudrait configurer explicitement des règles d'action comme ON DELETE CASCADE (supprime aussi les étudiants) ou ON DELETE SET NULL (met la clé étrangère des étudiants à NULL).

■ Exercice & Démonstration d'Application

Énoncé : On considère le schéma relationnel de l'IUT de Bayonne suivant : - Etudiant (id_etudiant, nom, prenom, #id_groupe) - Groupe (id_groupe, nom_groupe, promo) Écrire la requête SQL d'extraction conforme à la norme ISO/ANSI permettant d'afficher le nom et le prénom des étudiants appartenant à la promotion 'BUT1-INFO' et inscrits dans le groupe nommé 'A1'.

Démarche de résolution :
Étapes méthodologiques de résolution pas à pas : 1. Identification des tables à interroger : La requête nécessite des informations sur les étudiants (nom, prénom) et sur les groupes (promo, nom_groupe). Il faut donc les tables 'Etudiant' et 'Groupe'. 2. Attribution d'alias clairs : On attribue l'alias 'e' à la table Etudiant et l'alias 'g' à la table Groupe pour simplifier la lecture et éviter tout risque d'ambiguïté. 3. Écriture de la jointure explicite ANSI : La clé étrangère 'e.id_groupe' fait référence à la clé primaire 'g.id_groupe'. On écrit donc : `FROM Etudiant e INNER JOIN Groupe g ON e.id_groupe = g.id_groupe`. 4. Application des filtres de sélection dans la clause WHERE : On applique la double condition logique demandée : `g.promo = 'BUT1-INFO' AND g.nom_groupe = 'A1'`. 5. Spécification de la projection dans le SELECT : On sélectionne uniquement les attributs demandés par l'énoncé : `SELECT e.nom, e.prenom`.

```
SELECT e.nom, e.prenom
FROM Etudiant e
INNER JOIN Groupe g ON e.id_groupe = g.id_groupe
WHERE g.promo = 'BUT1-INFO'
  AND g.nom_groupe = 'A1';
```

Résultat : La requête retourne la liste exacte des noms et prénoms des étudiants inscrits dans le groupe A1 de BUT1-INFO. La syntaxe est conforme à la norme ANSI, évite le produit cartésien implicite et présente une structure optimale pour les moteurs d'exécution SQL.

■ Auto-Évaluation Flash d'Examen

Q : Quelles sont les deux contraintes fondamentales obligatoires que doit respecter toute Clé Primaire (Primary Key) ?

R : Une Clé Primaire doit obligatoirement respecter l'unicité stricte (aucun doublon autorisé dans la table) et la non-nullité (la valeur NULL est strictement interdite).

Q : Pourquoi la syntaxe 'FROM TableA, TableB WHERE TableA.id_b = TableB.id' fait-elle perdre des points en partiel de R1.05 ?

R : Parce qu'il s'agit d'une syntaxe obsolète basant la jointure sur un produit cartésien implicite. La norme ISO/ANSI exige l'utilisation de la syntaxe explicite 'INNER JOIN ... ON', qui sépare la structure des tables du filtrage des lignes.

Q : Qu'est-ce que la contrainte d'intégrité référentielle et quel est son rôle principal ?

R : C'est une règle automatisée par le SGBD qui garantit qu'une Clé Étrangère dans une table fille pointe systématiquement vers une Clé Primaire existante dans la table mère, empêchant ainsi la création de données orphelines ou incohérentes.